

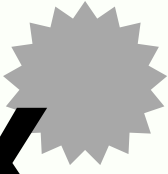
January 2024

Com. Sc. Project Synopsis

CAMP (Crime Analysis and
Monitoring Portal)

Prepared by: Tanishq Som

INDEX



INTRODUCTION	02
---------------------	-----------

MISSION & VISION	02
-----------------------------	-----------

SYSTEM ARCHITECTURE	03
----------------------------	-----------

DATABASE	05
-----------------	-----------

BACKEND	07
----------------	-----------

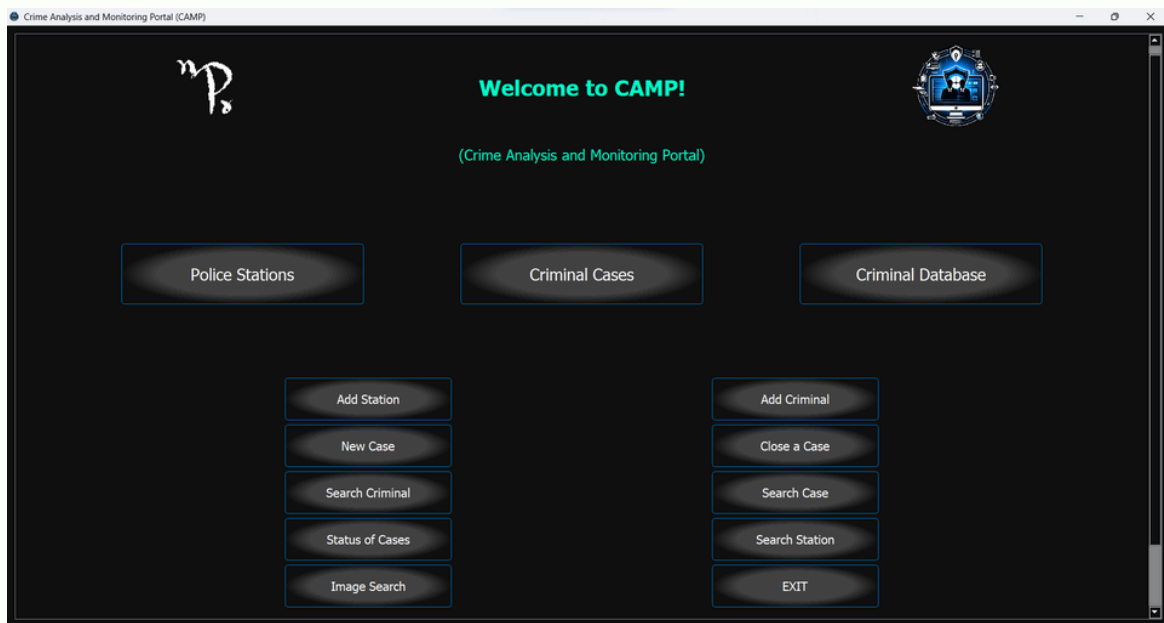
AI INTEGRATION	10
-----------------------	-----------

KEY FEATURES	11
---------------------	-----------

CONCLUSION	12
-------------------	-----------

Introduction

Crime Analysis and Monitoring Portal, abbreviated as CAMP, is a robust application designed to enhance the efficiency of law enforcement agencies in managing, analyzing, and recording critical crime-related information. Developed in Python with PyQt5 for the graphical interface, CAMP integrates seamlessly with MySQL for secure and organized data storage.



Mission and Vision

Our vision is to revolutionize the landscape of crime analysis and management by fostering a technologically advanced, user-friendly, and interconnected platform. We envision CAMP as a cornerstone in the digital transformation of law enforcement, equipping agencies with the tools they need to proactively address crime, protect citizens, and build trust within the communities they serve. Through continuous innovation, we aim to set new standards in the intersection of technology and public safety, making a lasting impact on the future of crime prevention and resolution.

System Architecture

The system architecture defining the major components, their interactions, and the flow of data within our Crime Management project.

Here's a high-level overview of the system architecture:

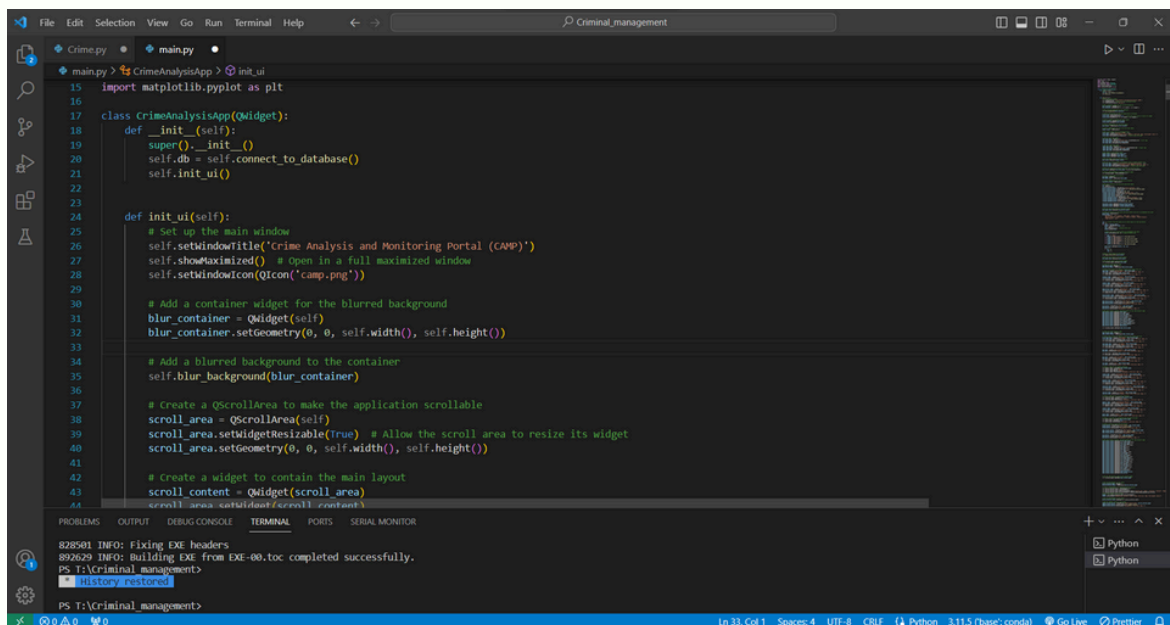
User Interface (UI):

- Developed using PyQt5 for the desktop application.
- The UI consists of multiple pages for various functionalities such as managing police stations, criminals, criminal cases, etc.



Backend Logic:

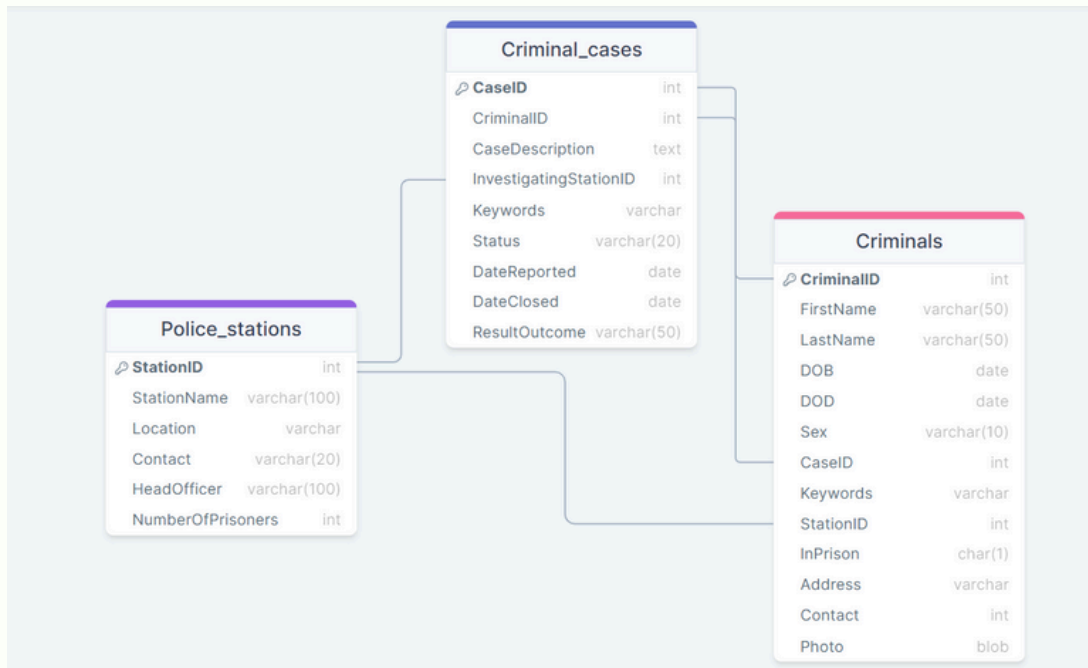
- Python functions handle the business logic of the application.
- Functions for adding, updating, and retrieving data from the database.
- Image processing (e.g., converting images to binary) and other utility functions.



```
File Edit Selection View Go Run Terminal Help
CrimeAnalysisApp
main.py
CrimeAnalysisApp > init_ui
15 import matplotlib.pyplot as plt
16
17 class CrimeAnalysisApp(QWidget):
18     def __init__(self):
19         super().__init__()
20         self.db = self.connect_to_database()
21         self.init_ui()
22
23     def init_ui(self):
24         # Set up the main window
25         self.setWindowTitle('Crime Analysis and Monitoring Portal (CAMP)')
26         self.showMaximized() # Open in a full maximized window
27         self.setWindowIcon(QIcon('camp.png'))
28
29         # Add a container widget for the blurred background
30         blur_container = QWidget(self)
31         blur_container.setGeometry(0, 0, self.width(), self.height())
32
33         # Add a blurred background to the container
34         self.blur_background(blur_container)
35
36         # Create a QScrollArea to make the application scrollable
37         scroll_area = QScrollArea(self)
38         scroll_area.setWidgetResizable(True) # Allow the scroll area to resize its widget
39         scroll_area.setGeometry(0, 0, self.width(), self.height())
40
41         # Create a widget to contain the main layout
42         scroll_content = QWidget(scroll_area)
43         scroll_area.setWidget(scroll_content)
44
45 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
828501 INFO: Fixing EXE headers
892629 INFO: Building EXE from EXE-00.toc completed successfully.
PS T:\Criminal_management>
History restored
PS T:\Criminal_management>
```

Database Layer:

- MySQL database is used to store and manage data.
- Tables include police_stations, criminals, and criminal_cases.
- Establish proper relationships between tables using foreign keys.



External Libraries and Tools:

- PyQt5: Used for creating the graphical user interface.
- MySQL Connector: Enables communication with the MySQL database.
- OpenCV and face_recognition: For image-related processing and facial recognition if implemented.

Notification Module:

- PyQt5's QMessageBox for displaying notifications to users.

File Handling:

- Handling image files, opening file dialogs for browsing, and managing file paths.

Database

The Database for CAMP application is "crime_management" MySQL Database.

```

MySQL 8.0 Command Line Cli  x  +  v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.34 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE crime_management;
Query OK, 1 row affected (0.01 sec)

```

The "crime_management" database contains three tables to store data of police stations, criminals, and all criminal cases ever filed.

```

mysql> create table police_stations (
    -> StationID INT PRIMARY KEY AUTO_INCREMENT,
    -> StationName VARCHAR(100),
    -> Location VARCHAR(255),
    -> Contact VARCHAR(20),
    -> HeadOfficer VARCHAR(100),
    -> NumberOfPrisoners INT
    -> );
Query OK, 0 rows affected (0.02 sec)

```

Command to create table "police_stations"

```
mysql> DESCRIBE police_stations;
```

Field	Type	Null	Key	Default	Extra
StationID	int	NO	PRI	NULL	auto_increment
StationName	varchar(100)	YES		NULL	
Location	varchar(255)	YES		NULL	
Contact	varchar(20)	YES		NULL	
HeadOfficer	varchar(100)	YES		NULL	
NumberOfPrisoners	int	YES		NULL	

6 rows in set (0.00 sec)

Description of table "police_stations"

```
mysql> CREATE TABLE Criminal_Cases (
-> CaseID INT PRIMARY KEY AUTO_INCREMENT,
-> CriminalID INT,
-> CaseDescription TEXT,
-> InvestigatingStationID INT,
-> Keywords VARCHAR(255),
-> Status VARCHAR(20),
-> DateReported DATE,
-> DateClosed DATE,
-> ResultOutcome VARCHAR(50),
-> FOREIGN KEY (InvestigatingStationID) REFERENCES Police_Stations(StationID)
-> );
Query OK, 0 rows affected (0.07 sec)
mysql>
```

Command to create table "Criminal_cases"

```
mysql> DESCRIBE criminal_cases;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| CaseID | int | NO | PRI | NULL | auto_increment |
| CriminalID | int | YES | | NULL | |
| CaseDescription | text | YES | | NULL | |
| InvestigatingStationID | int | YES | MUL | NULL | |
| Keywords | varchar(255) | YES | | NULL | |
| Status | varchar(20) | YES | | NULL | |
| DateReported | date | YES | | NULL | |
| DateClosed | date | YES | | NULL | |
| ResultOutcome | varchar(50) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

Description of table "criminal_cases"

```
mysql> DESCRIBE criminals;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| CriminalID | int | NO | PRI | NULL | auto_increment |
| FirstName | varchar(50) | YES | | NULL | |
| LastName | varchar(50) | YES | | NULL | |
| DOB | date | YES | | NULL | |
| DOD | date | YES | | NULL | |
| Sex | varchar(10) | YES | | NULL | |
| CaseID | int | YES | MUL | NULL | |
| Keywords | varchar(255) | YES | | NULL | |
| StationID | int | YES | MUL | NULL | |
| InPrison | char(1) | YES | | NULL | |
| Address | varchar(255) | YES | | NULL | |
| Contact | varchar(20) | YES | | NULL | |
| Photo | longblob | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
13 rows in set (0.00 sec)
```

Description of table "criminals"

Backend

"The backend of CAMP is responsible for managing the application's data, handling database interactions, and executing business logic. It is implemented using Python and utilizes the PyQt5 library for the graphical user interface (GUI) components. The primary functionalities of the backend include database connectivity, CRUD (Create, Read, Update, Delete) operations for police stations, criminals, and criminal cases, as well as additional features for case status updates and criminal deletion."

Database Connectivity:

- The backend establishes a connection to a MySQL database using the [mysql.connector library](#). Database configuration parameters such as host, user, password, and database name are specified during the connection setup.

```
!pip install mysql-connector-python
----- 14.1/14.2 MB 143.4 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.4 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.4 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.4 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.4 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.0 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.0 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.0 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.0 kB/s eta 0:00:01
----- 14.1/14.2 MB 143.0 kB/s eta 0:00:01
----- 14.1/14.2 MB 141.7 kB/s eta 0:00:01
----- 14.1/14.2 MB 141.7 kB/s eta 0:00:01
----- 14.1/14.2 MB 141.7 kB/s eta 0:00:01
----- 14.2/14.2 MB 141.9 kB/s eta 0:00:01
----- 14.2/14.2 MB 141.9 kB/s eta 0:00:01
----- 14.2/14.2 MB 141.9 kB/s eta 0:00:01
----- 14.2/14.2 MB 141.8 kB/s eta 0:00:01
----- 14.2/14.2 MB 142.3 kB/s eta 0:00:01
----- 14.2/14.2 MB 142.3 kB/s eta 0:00:01
----- 14.2/14.2 MB 142.6 kB/s eta 0:00:01
----- 14.2/14.2 MB 142.1 kB/s eta 0:00:00
Installing collected packages: protobuf, mysql-connector-python
Successfully installed mysql-connector-python-8.2.0 protobuf-4.21.12
```


Police Stations Management:

- The backend provides functionality to add new police stations to the database. This includes gathering information such as station name, location, contact details, head officer, and the number of prisoners. The data is then inserted into the `police_stations` table.

```
[42]: def add_station(name, location=None, contact=None, head_officer=None, num_prisoners=None):
      try:
          insert_query = "INSERT INTO police_stations (StationName, Location, Contact, HeadOfficer, NumberOfPrisoners) VALUES (%s, %s, %s, %s, %s)"
          data = (name, location, contact, head_officer, num_prisoners)
          cursor.execute(insert_query, data)
          db.commit()
          print(f'{name} added to the database successfully!')
      except:
          print("Operation Failed! Please try again later.")
```

Python Function to handle queries for `police_stations` table.

The left screenshot displays a table of police stations:

Station ID	Name	Location	Contact	Head Officer	No. of Prisoners
1	Northland Street Station	Northland Street, Northland	744723456	John Doe	10
2	South Street Station	South Street, Southland	744723457	John Doe	12

The right screenshot shows a form to add a new police station:

Welcome to CAMP!
(Crime Analysis and Monitoring Portal)

Station Name:

Location:

Contact:

Head Officer:

Number of Prisoners:

Buttons: Update Station Data, Delete Station Data, Back

UX To view, delete or update Police Stations data from database.

Form to add a new Police Station to the database.

Criminals Management:

- Adding New Criminals:** The backend allows the addition of new criminals to the database. Information such as first name, last name, date of birth, date of death (if applicable), gender, associated case ID, keywords, station ID, prison status, address, contact details, and a photo are collected and stored in the criminals table.
- Updating Criminals:** The backend supports updating existing criminal records, including modifications to personal details, case associations, or prison status.

```
[44]: def add_criminal(FirstName, LastName=None, DOB=None, DOD=None, Sex=None, CaseID=None, Keywords=None, StationID=None, InPrison=None, Address=None, Contact=None):
    try:
        insert_query = "INSERT INTO criminals (FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID, InPrison, Address, Contact, Photo) VALUES"
        data = (FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID, InPrison, Address, Contact, Photo)
        cursor.execute(insert_query, data)
        db.commit()
        print("Criminal added successfully!")
    except:
        print("Operation Failed! Please try again later.")
```

Python function to handle queries related to Criminals table.

UX To view, delete or update Criminals data from database.

Form to add a new Criminal to the database.

Criminal Cases Management:

- **Creating New Cases:** The backend enables the addition of new criminal cases to the criminal_cases table. Information such as case name, status (Pending or Closed), and a detailed description of the case is collected and stored.
- **Updating Case Status:** A feature is provided to update the status of a criminal case, marking it as either 'Pending' or 'Closed.'

```
[80]: def add_case(CriminalID=None, CaseDescription=None, InvestigatingStationID=None, Keywords=None, Status="Pending", DateReported=None):
    try:
        if DateReported is None:
            DateReported = datetime.now().strftime('%Y-%m-%d')
        insert_query = "INSERT INTO criminal_cases (CriminalID, CaseDescription, InvestigatingStationID, Keywords, Status, DateReported, DateClosed, Result)"
        data = (CriminalID, CaseDescription, InvestigatingStationID, Keywords, Status, DateReported, None, None)
        cursor.execute(insert_query, data)
        db.commit()
        print("Case filed successfully!")
    except:
        print("Operation Failed! Please try again later.")
```

Python Function to handle queries for criminal_cases table.

UX To view, delete or update Criminal cases data from database.

Form to add a new Criminal Case to the database.

Image Search Feature with AI Integration:

CAMP's Image Search feature utilizes advanced AI for facial recognition, aiding law enforcement in identifying and apprehending criminals. Here's an overview:

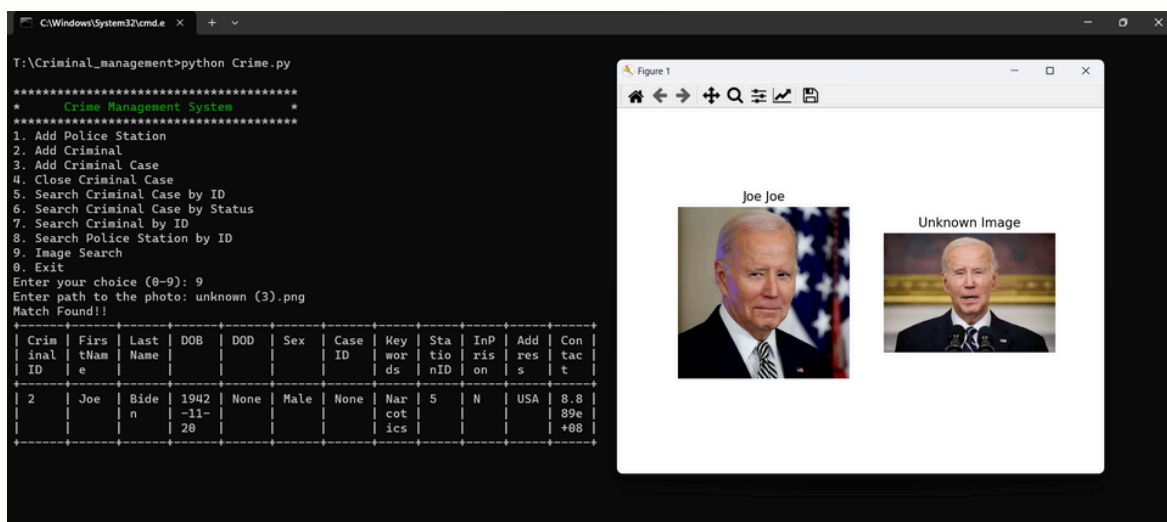
- **User Input:** Users provide an image with a face, sourced from surveillance footage, photographs, or sketches.
- **Algorithm:** The application's face recognition algorithm processes the image, extracting key facial features.
- **Database Query:** The extracted features are compared against a criminal database for potential matches.
- **Result Display:** Successful matches are presented with relevant details, aiding investigations.

```
[175]: def img_search(bphoto):
    unknown_image_data = convert_binary_to_image(bphoto)
    unknown_image_array = np.array(unknown_image_data)
    unknown_encoding = face_recognition.face_encodings(unknown_image_array)[0]

    query = f"SELECT photo, CriminalID, FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID, InPrison, Address, Contact FROM criminals"
    cursor.execute(query)
    result = cursor.fetchall()

    if result:
        for match in result:
            known_image_data = convert_binary_to_image(match[0])
            known_image_array = np.array(known_image_data)
            known_encoding = face_recognition.face_encodings(known_image_array)[0]
            results = face_recognition.compare_faces([known_encoding], unknown_encoding)
            if results[0]:
                print("Match Found!!")
                header = ["CriminalID", "FirstName", "LastName", "DOB", "DOD", "Sex", "CaseID", "Keywords", "StationID", "InPrison", "Address", "Contact"]
                data = [list(map(str, match[1:]))]
                print_table(header, data)
                # Display images
                display_images(known_image_array, unknown_image_array, str(data[0][1] + ' ' + data[0][2]), "Unknown Image")
                break
            else:
                if match == result[-1]:
                    print("No matching photo found.")
        else:
            print("No matching photo found.")
```

Artificial Intelligence Face Recognition Algorithm to process an image.



Feature in Action, upon giving an unknown image it gave us a match from database. [information is used for demo purposes only and not true :)]

Key Features

1. Police Stations:

- Add, view, and edit police station information.
- Includes station name, location, contact, head officer, and prisoner count.

2. Criminals Database:

- Add, view, and edit criminal records.
- Capture details like name, DOB, DOD, gender, case ID, keywords, and in-prison status.
- Browse and attach criminal photos.

3. Criminal Cases:

- Add, view, and edit criminal cases.
- Capture case ID, associated criminal ID, description, status (closed/pending), and station ID.
- Change case status between "Closed" and "Pending."

4. User Interface:

- Dark-themed, user-friendly interface.
- Navigation through buttons and stacked widget.
- Notifications for successful operations and error messages.

5. Database Integration:

- Utilizes MySQL for data storage and retrieval.
- Supports add, update, and delete operations.

6. Image Processing:

- Allows attaching criminal photos.
- Supports browsing and attaching photos from the local file system.

7. Customization:

- Customizable application icon.
- Generates standalone executable (.exe) for easy distribution.

Technical Stack:

- Language: Python
- Framework: PyQt5
- Database: MySQL
- Libraries: OpenCV, NumPy, Pillow, PyInstaller



```
#!/usr/bin/env python

import mysql.connector
import io
import base64
from PIL import Image, ImageTk
import cv2
import numpy as np
from datetime import datetime
from texttable import Texttable
import face_recognition
import matplotlib.pyplot as plt
from prettytable import PrettyTable
from colorama import Fore, Style, init
init(autoreset=True)

# Application Config Functions
def convert_image_to_binary(image_path):
    with open(image_path, "rb") as file:
        binary_data = file.read()
    return binary_data

def convert_binary_to_image(binary_data):
    image = Image.open(io.BytesIO(binary_data))
    return image

def display_images(known_image_array, unknown_image_array, known_title, unknown_title):
    plt.subplot(1, 2, 1)
    plt.imshow(known_image_array)
    plt.title(known_title)
    plt.axis('off')
    plt.subplot(1, 2, 2)
    plt.imshow(unknown_image_array)
    plt.title(unknown_title)
    plt.axis('off')
    plt.show()

def print_table(header, data):
    t = Texttable()
    t.add_row(header)
    for row in data:
        t.add_row(row)
    print(t.draw())

#Database Configuration
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="iamts@84",
    database="crime_management"
)
cursor = db.cursor()
```



```
def add_station(name, location=None, contact=None, head_officer=None, num_prisoners=None):
    try:
        insert_query = "INSERT INTO police_stations (StationName, Location, Contact,
HeadOfficer, NumberOfPrisoners) VALUES (%s, %s, %s, %s, %s)"
        data = (name, location, contact, head_officer, num_prisoners)
        cursor.execute(insert_query, data)
        db.commit()
        print(f"{name} added to the database successfully!")
    except:
        print("Operation Failed! Please try again later.")

def add_criminal(FirstName, LastName=None, DOB=None, DOD=None, Sex=None, CaseID=None,
Keywords=None, StationID=None, InPrison=None, Address=None, Contact=None, Photo=None):
    try:
        insert_query = "INSERT INTO criminals (FirstName, LastName, DOB, DOD, Sex, CaseID,
Keywords, StationID, InPrison, Address, Contact, Photo) VALUES (%s, %s, %s, %s, %s, %s,
%s, %s, %s, %s, %s, %s)"
        data = (FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID, InPrison,
Address, Contact, Photo)
        cursor.execute(insert_query, data)
        db.commit()
        print("Criminal added successfully!")
    except:
        print("Operation Failed! Please try again later.")

def add_case(CriminalID=None, CaseDescription=None, InvestigatingStationID=None,
Keywords=None, Status="Pending", DateReported=None):
    try:
        if DateReported is None:
            DateReported = datetime.now().strftime('%Y-%m-%d')
        insert_query = "INSERT INTO criminal_cases (CriminalID, CaseDescription,
InvestigatingStationID, Keywords, Status, DateReported, DateClosed, ResultOutcome) VALUES
(%s, %s, %s, %s, %s, %s, %s, %s)"
        data = (CriminalID, CaseDescription, InvestigatingStationID, Keywords, Status,
DateReported, None, None)
        cursor.execute(insert_query, data)
        db.commit()
        print("Case filed successfully!")
    except:
        print("Operation Failed! Please try again later.")

def search_criminal_case_by_status(status):
    query = f"SELECT * FROM criminal_cases WHERE Status = '{status}'"
    cursor.execute(query)
    result = cursor.fetchall()
    if result:
        header = ["CaseID", "CriminalID", "CaseDescription", "InvestigatingStationID",
"Keywords", "Status", "DateReported", "DateClosed", "ResultOutcome"]
        data = [list(map(str, row)) for row in result]
        print_table(header, data)
    else:
        print(f"No criminal cases with status '{status}' found.")
```



```
def close_case(CaseID, ResultOutcome, DateClosed=None):
    try:
        if DateClosed is None:
            DateClosed = datetime.now().strftime('%Y-%m-%d')
            update_query = "UPDATE criminal_cases SET Status = 'Closed', DateClosed = %s,
ResultOutcome = %s WHERE CaseID = %s"
            update_data = (DateClosed, ResultOutcome, CaseID)
            cursor.execute(update_query, update_data)
            db.commit()
            print("Case Closed successfully!")
            query = f"SELECT * FROM criminal_cases WHERE CaseID = {CaseID}"
            cursor.execute(query)
            case = cursor.fetchall()
            t = Texttable()
            t.add_rows([["CaseID", "CriminalID", "CaseDescription", "InvestigatingStationID",
"Keywords", "Status", "DateReported", "DateClosed", "ResultOutcome"], case[0]])
            print(t.draw())
        except:
            print("Operation Failed! Please try again later.")

def search_criminal_case_by_id(case_id):
    query = f"SELECT * FROM criminal_cases WHERE CaseID = {case_id}"
    cursor.execute(query)
    result = cursor.fetchall()
    if result:
        header = ["CaseID", "CriminalID", "CaseDescription", "InvestigatingStationID",
"Keywords", "Status", "DateReported", "DateClosed", "ResultOutcome"]
        data = [list(map(str, result[0]))]
        print_table(header, data)
    else:
        print("No matching criminal case found.")

def search_criminal_by_id(criminal_id):
    query = f"SELECT * FROM criminals WHERE CriminalID = {criminal_id}"
    cursor.execute(query)
    result = cursor.fetchall()
    if result:
        header = ["CriminalID", "FirstName", "LastName", "DOB", "DOD", "Sex", "CaseID",
"Keywords", "StationID", "InPrison", "Address", "Contact"]
        data = [list(map(str, result[0][:11]))]
        print_table(header, data)
    else:
        print("No matching criminal found.")

def search_police_station_by_id(station_id):
    query = f"SELECT * FROM police_stations WHERE StationID = {station_id}"
    cursor.execute(query)
    result = cursor.fetchall()
    if result:
        header = ["StationID", "StationName", "Location", "Contact", "HeadOfficer",
"NumberOfPrisoners"]
        data = [list(map(str, result[0]))]
        print_table(header, data)
    else:
        print("No matching police station found.")
```



```
def img_search(bphoto):
    unknown_image_data = convert_binary_to_image(bphoto)
    unknown_image_array = np.array(unknown_image_data)
    unknown_encoding = face_recognition.face_encodings(unknown_image_array)[0]
    query = f"SELECT photo, CriminalID, FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID, InPrison, Address, Contact FROM criminals"
    cursor.execute(query)
    result = cursor.fetchall()
    if result:
        for match in result:
            known_image_data = convert_binary_to_image(match[0])
            known_image_array = np.array(known_image_data)
            known_encoding = face_recognition.face_encodings(known_image_array)[0]
            results = face_recognition.compare_faces([known_encoding], unknown_encoding)
            if results[0]:
                print("Match Found!!")
                header = ["CriminalID", "FirstName", "LastName", "DOB", "DOD", "Sex", "CaseID", "Keywords", "StationID", "InPrison", "Address", "Contact"]
                data = [list(map(str, match[1:]))]
                print_table(header, data)
                # Display images
                display_images(known_image_array, unknown_image_array, str(data[0][1] + ' ' + data[0][2]), "Unknown Image")
                break
            else:
                if match == result[-1]:
                    print("No matching photo found.")
    else:
        print("No matching photo found.")

if __name__ == '__main__':
    while True:
        print("\n" + Fore.WHITE + "*****")
        print("*" + Fore.GREEN + "    Crime Management System    " + Fore.WHITE + "*")
        print("*****")
        print("1. Add Police Station")
        print("2. Add Criminal")
        print("3. Add Criminal Case")
        print("4. Close Criminal Case")
        print("5. Search Criminal Case by ID")
        print("6. Search Criminal Case by Status")
        print("7. Search Criminal by ID")
        print("8. Search Police Station by ID")
        print("9. Image Search")
        print("0. Exit")
        choice = input("Enter your choice (0-9): ")
        if choice == '1':
            # Add Police Station
            name = input("Enter Station Name: ")
            location = input("Enter Location: ")
            contact = input("Enter Contact: ")
            head_officer = input("Enter Head Officer: ")
            num_prisoners = input("Enter Number of Prisoners: ")
            add_station(name, location, contact, head_officer, num_prisoners)
```




```
elif choice == '2':
    # Add Criminal
    FirstName = input("Enter First Name: ")
    LastName = input("Enter Last Name: ")
    DOB = input("Enter Date of Birth (YYYY-MM-DD): ")
    DOD = input("Enter Date of Death (YYYY-MM-DD): ")
    Sex = input("Enter Sex: ")
    CaseID = input("Enter Case ID: ")
    Keywords = input("Enter Keywords: ")
    StationID = input("Enter Station ID: ")
    InPrison = input("Is the criminal in prison? (Y/N): ")
    Address = input("Enter Address: ")
    Contact = input("Enter Contact: ")
    Photo = input("Enter path to Photo: ")
    add_criminal(FirstName, LastName, DOB, DOD, Sex, CaseID, Keywords, StationID,
    InPrison, Address, Contact, convert_image_to_binary(Photo))

elif choice == '3':
    # Add Criminal Case
    CriminalID = input("Enter Criminal ID: ")
    CaseDescription = input("Enter Case Description: ")
    InvestigatingStationID = input("Enter Investigating Station ID: ")
    Keywords = input("Enter Keywords: ")
    add_case(CriminalID, CaseDescription, InvestigatingStationID, Keywords)

elif choice == '4':
    # Close Criminal Case
    CaseID = input("Enter Case ID to close: ")
    ResultOutcome = input("Enter Result Outcome: ")
    close_case(CaseID, ResultOutcome)

elif choice == '5':
    # Search Criminal Case by ID
    case_id = input("Enter Case ID: ")
    search_criminal_case_by_id(case_id)

elif choice == '6':
    # Search Criminal Case by Status
    status = input("Enter Status: ")
    search_criminal_case_by_status(status)

elif choice == '7':
    # Search Criminal by ID
    criminal_id = input("Enter Criminal ID: ")
    search_criminal_by_id(criminal_id)

elif choice == '8':
    # Search Police Station by ID
    station_id = input("Enter Station ID: ")
    search_police_station_by_id(station_id)
```



```
elif choice == '9':  
    # Image Search  
    photo_path = input("Enter path to the photo: ")  
    img_search(convert_image_to_binary(photo_path))  
  
elif choice == '0':  
    # Exit  
    print("\n" + Fore.RED + "Exiting Crime Management System. Goodbye!")  
    break  
  
else:  
    print("\n" + Fore.RED + "Invalid choice. Please enter a number between 0 and 9.")
```

Above is the code for CLI (Command Line Interface) of CAMP, for full code with GUI (Graphical User Interface) refer to the GitHub profile of maintainer and owner of this project, Tanishq Som.

Link to GitHub Repository: <https://github.com/Tanishq84/CAMP>

Alternatively, you can Scan the QR code given below,



Conclusion

CAMP is a comprehensive solution that not only streamlines the management of police stations, criminals, and criminal cases but also prioritizes user experience, data security, and customization. This application stands as a testament to the synergy between technology and law enforcement, enhancing the capabilities of agencies in addressing and resolving criminal activities effectively.

It is a versatile tool for efficient crime data management, offering a user-friendly interface, database integration, and image processing capabilities. It provides a comprehensive solution for law enforcement agencies to handle police stations, criminals, and criminal cases effectively.

